

# AgentKit

A library-first, VM-free agent harness & orchestration platform — low-level functional & technical design.

|              |                                                                                |
|--------------|--------------------------------------------------------------------------------|
| COMPONENT    | AgentKit core library (agentkit)                                               |
| DISTRIBUTION | PyPI wheel · <code>pip install agentkit[...]</code>                            |
| RUNTIME      | Python 3.11+ (asyncio) · in-process SDK or local app via <code>.serve()</code> |
| CONSTRAINTS  | No VMs · no separate orchestrator · durable state via embedded DB              |
| STATUS       | Draft v0.1                                                                     |
| AUDIENCE     | Engineers implementing or extending the platform                               |

This document specifies both **what** the platform must do (functional design) and **how** it is built (technical design), at signature and schema granularity. Each chapter opens with an overview and is illustrated with figures.

# Contents

---

|           |                                                            |    |
|-----------|------------------------------------------------------------|----|
| <b>0</b>  | Document control .....                                     | 3  |
| <b>1</b>  | Functional design .....                                    | 4  |
| <b>2</b>  | System architecture .....                                  | 6  |
| <b>3</b>  | Low-level technical design — core modules .....            | 8  |
| <b>4</b>  | Protocol modules .....                                     | 14 |
| <b>5</b>  | Memory (agentkit/memory/) .....                            | 16 |
| <b>6</b>  | Tools (agentkit/tools/) .....                              | 17 |
| <b>7</b>  | Observability & evaluation (agentkit/observability/) ..... | 19 |
| <b>8</b>  | Identity & secrets (agentkit/identity/) .....              | 21 |
| <b>9</b>  | Execution lifecycle (end-to-end sequences) .....           | 22 |
| <b>10</b> | Error handling .....                                       | 24 |
| <b>11</b> | Security model .....                                       | 25 |
| <b>12</b> | Configuration reference .....                              | 26 |
| <b>13</b> | Packaging & distribution .....                             | 28 |
| <b>14</b> | Extensibility points .....                                 | 29 |
| <b>15</b> | Build milestones .....                                     | 30 |
| <b>16</b> | Glossary .....                                             | 31 |
| <b>17</b> | Open questions .....                                       | 32 |

# 0 Document control

## OVERVIEW

Establishes the identity and ground rules of the system: what AgentKit is, how it is distributed, the runtime it targets, and the hard constraints that shape every later decision. The definitions introduced here form the vocabulary used throughout the document.

| Field                            | Value                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------|
| Component                        | AgentKit core library (agentkit)                                                  |
| Distribution                     | PyPI wheel; <code>pip install agentkit[...]</code>                                |
| Runtime targets                  | Python 3.11+ (asyncio)                                                            |
| Execution modes                  | (a) in-process SDK, (b) local app via <code>.serve()</code>                       |
| External managed deps (optional) | Langfuse (obs/eval), E2B (untrusted sandbox), Qdrant/Postgres (prod stores)       |
| Hard constraints                 | No VMs/microVMs; no separate workflow orchestrator; durable state via embedded DB |

## 0.1 Definitions

- **Agent** — a configured reasoning unit: model + tools + memory + protocol bindings.
- **Flow** — a graph composing agents/functions with branching, parallelism, loops.
- **Tool** — an invokable capability (local function, MCP tool, or A2A peer call).
- **Session** — an isolated, durable execution context keyed by `thread_id`.
- **Run** — one invocation of an Agent/Flow within a session, producing a `RunResult`.
- **Checkpoint** — a persisted snapshot of graph state enabling resume.

# 1 Functional design

## OVERVIEW

Defines what the platform must do before any code is written: explicit goals and non-goals, the actors who interact with it, and a numbered catalogue of functional and non-functional requirements. The use cases translate those requirements into concrete journeys the technical design must satisfy.

## 1.1 Goals / Non-goals

### Goals

- G1 — Build agents as importable Python objects; no infra to provision for dev.
- G2 — The same object runs in-process (`run`) or as a local service (`serve`).
- G3 — First-class MCP (consume + expose) and A2A (peer + expose).
- G4 — Durable, resumable runs from an embedded DB (SQLite default).
- G5 — Built-in observability and evaluation through a single Langfuse hub.
- G6 — Pluggable memory with embedded defaults and production backends.
- G7 — Browser automation built in; code/computer execution with explicit trust boundaries.
- G8 — Publishable and consumable as an SDK by downstream teams.

### Non-goals

- N1 — Hardware-level isolation of untrusted code (delegated to E2B when required).
- N2 — Multi-tenant SaaS control plane, billing, quota (out of core scope).
- N3 — A bundled foundation model (BYO model + key).
- N4 — A visual no-code builder in core (can be layered via Langflow/Flowise).

## 1.2 Actors

| Actor               | Description                          | Primary interactions               |
|---------------------|--------------------------------------|------------------------------------|
| Agent author        | Engineer defining agents/flows/tools | SDK API, agent spec YAML           |
| Platform operator   | Runs served agents                   | serve, CLI, config, health/metrics |
| Calling agent (A2A) | Remote agent invoking a served agent | /a2a, agent card                   |
| Calling app (MCP)   | MCP client invoking exposed tools    | /mcp                               |
| Eval engineer       | Builds datasets, judges quality      | Langfuse datasets, eval runner     |

## 1.3 Functional requirements

| ID   | Requirement                                                                                                                                                |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1 | Define an agent with model, instructions, tools, memory, MCP/A2A bindings via constructor or YAML.                                                         |
| FR-2 | Execute an agent synchronously ( <code>run</code> ) and asynchronously ( <code>arun</code> ), with optional token streaming.                               |
| FR-3 | Compose agents/functions into a flow with sequential, conditional, parallel, and loop control.                                                             |
| FR-4 | Serve any Agent/Flow as a local app exposing <code>/invoke</code> , <code>/mcp</code> , <code>/a2a</code> , <code>/health</code> , <code>/metrics</code> . |

| ID    | Requirement                                                                                          |
|-------|------------------------------------------------------------------------------------------------------|
| FR-5  | Consume external MCP servers (stdio/http/sse) and surface their tools to the agent.                  |
| FR-6  | Author and serve custom MCP servers from decorated Python functions.                                 |
| FR-7  | Publish an agent as an A2A peer (agent card + task lifecycle) and call other A2A peers.              |
| FR-8  | Persist run state to an embedded checkpointer; resume an interrupted run by <code>thread_id</code> . |
| FR-9  | Read/write short-term (session) and long-term (user/agent) memory with selectable strategies.        |
| FR-10 | Emit traces/spans and eval scores to Langfuse with no per-call boilerplate.                          |
| FR-11 | Run offline evals (dataset × metrics) and push scores to the same Langfuse project.                  |
| FR-12 | Drive a browser (navigate/click/type/extract/screenshot) as a built-in tool group.                   |
| FR-13 | Execute code via a trusted (subprocess) or untrusted (E2B) executor selectable per call.             |
| FR-14 | Drive a desktop (computer-use) only when explicitly enabled.                                         |
| FR-15 | Resolve outbound credentials per tool/server from a pluggable secret provider.                       |

## 1.4 Non-functional requirements

| ID    | Requirement                                                                                          |
|-------|------------------------------------------------------------------------------------------------------|
| NFR-1 | Zero-infra cold start: <code>Agent(...).run(...)</code> works with only a model key set.             |
| NFR-2 | Durable resume after process crash with no external orchestrator.                                    |
| NFR-3 | Concurrency via <code>asyncio</code> ; CPU-bound/risky work offloaded off the event loop.            |
| NFR-4 | Deterministic, typed errors (no bare exceptions across the public API).                              |
| NFR-5 | Backend swap (store/checkpointer/executor) without changing agent code.                              |
| NFR-6 | Extensible via entry-point plugins ( <code>agentkit.tools</code> , <code>agentkit.backends</code> ). |
| NFR-7 | Trace overhead < ~5% latency under default sampling.                                                 |

## 1.5 Key use cases

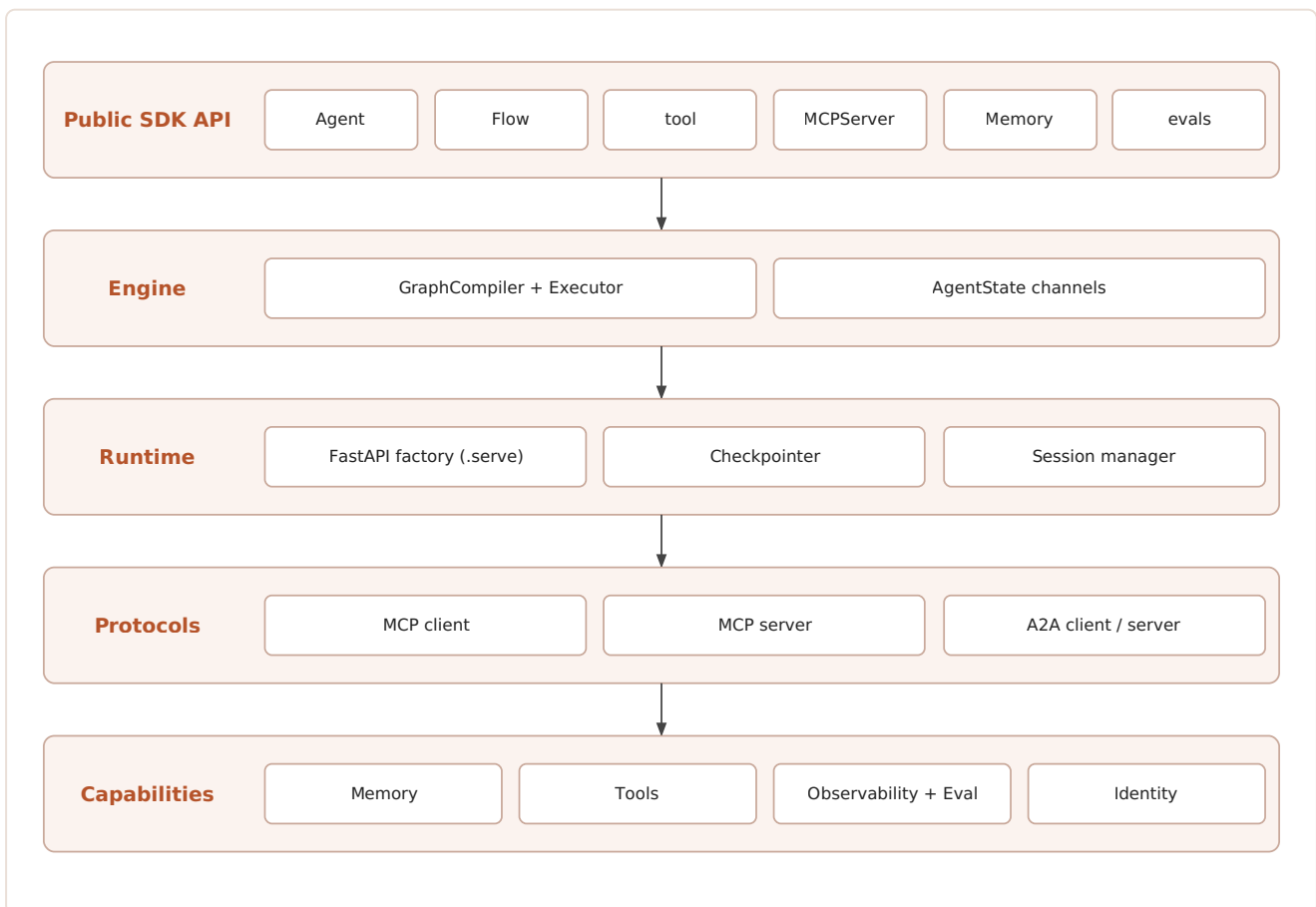
- **UC-1 Single agent, in-process** — author imports `Agent`, registers tools, calls `run`, gets a `RunResult` with messages, tool calls, and a trace URL.
- **UC-2 Served agent over MCP** — operator calls `serve`; an MCP client lists and invokes the agent's tools and an `invoke_agent` meta-tool.
- **UC-3 Multi-agent flow** — author composes researcher → writer → reviewer with a conditional review step; serves the flow as one app.
- **UC-4 A2A delegation** — a served researcher exposes `/a2a`; a remote planner sends it a task, streams progress, and collects artifacts.
- **UC-5 Durable long task** — a run is interrupted (crash/timeout); operator resumes by `thread_id` and the run continues from the last checkpoint.
- **UC-6 Eval loop** — eval engineer runs a Langfuse dataset through the agent, computes faithfulness/tool-correctness/task-success, and reviews scores in Langfuse.

## 2 System architecture

### OVERVIEW

Presents the layered structure of the system and the module inventory that realizes the requirements. It shows how the SDK surface, engine, runtime, protocols, and capability modules stack and depend on one another, and names the cross-cutting concerns every module honors.

### 2.1 Layered view



**Figure 1.** Layered architecture and module dependencies.

### 2.2 Module inventory

| Module       | Package path                   | Responsibility                           |
|--------------|--------------------------------|------------------------------------------|
| Agent core   | agentkit/agent.py              | Agent model, run loop, tool binding      |
| Flow engine  | agentkit/flow.py               | Graph composition + compilation          |
| Engine       | agentkit/engine/               | State channels, graph compiler, executor |
| Runtime app  | agentkit/runtime/app.py        | FastAPI factory, endpoints, lifespan     |
| Checkpointer | agentkit/runtime/checkpoint.py | Durable state persistence                |
| Session mgr  | agentkit/runtime/session.py    | Isolation, concurrency, locks            |

| Module        | Package path                | Responsibility                        |
|---------------|-----------------------------|---------------------------------------|
| MCP           | agentkit/protocols/mcp_*.py | Consume + expose MCP                  |
| A2A           | agentkit/protocols/a2a.py   | Peer + expose A2A                     |
| Memory        | agentkit/memory/            | Strategies, scopes, store adapters    |
| Tools         | agentkit/tools/             | browser / code / computer / registry  |
| Observability | agentkit/observability/     | Tracing + eval                        |
| Identity      | agentkit/identity/          | Secret providers, credential resolver |
| CLI           | agentkit/cli.py             | agentkit serve/run/eval               |

## 2.3 Cross-cutting concerns

- **Typing** — Pydantic v2 models for all public config and wire payloads.
- **Async-first** — every blocking integration has an async path; sync wrappers call `asyncio.run`.
- **Context propagation** — a `RunContext` (session\_id, trace\_id, deadline, secrets) threads through engine → tools → protocols via `contextvars`.
- **Config precedence** — explicit kwargs > YAML spec > env vars > defaults.

## 3 Low-level technical design — core modules

### OVERVIEW

The heart of the technical design: signature-level specifications for the Agent, the Flow engine, the serving runtime, the durable checkpointer, and the session manager. Each subsection gives the public types, the state it operates on, and the lifecycle that governs its behavior.

### 3.1 Agent core (agentkit/agent.py)

#### 3.1.1 Public model

```
class Agent(BaseModel):
    name: str
    model: str # provider/model id, BYO key
    instructions: str | None = None
    tools: list[Tool | Callable] = []
    mcp_servers: list[str | MCPRef] = [] # urls or refs (e.g. "browser-use")
    a2a_peers: list[str] = [] # agent-card urls
    memory: MemoryConfig | str | None = None # "mem0://local" shorthand
    observability: ObsConfig | str | None = "langfuse"
    max_steps: int = 20
    step_timeout_s: float = 120
    on_tool_error: Literal["surface", "raise"] = "surface"
    model_settings: ModelSettings = ModelSettings()

    # lifecycle
    def __post_init__(self) -> None: ... # normalize tools, resolve refs
    def register_tool(self, fn: Callable) -> Tool: ...
    def as_graph(self) -> CompiledGraph: ... # cached compile

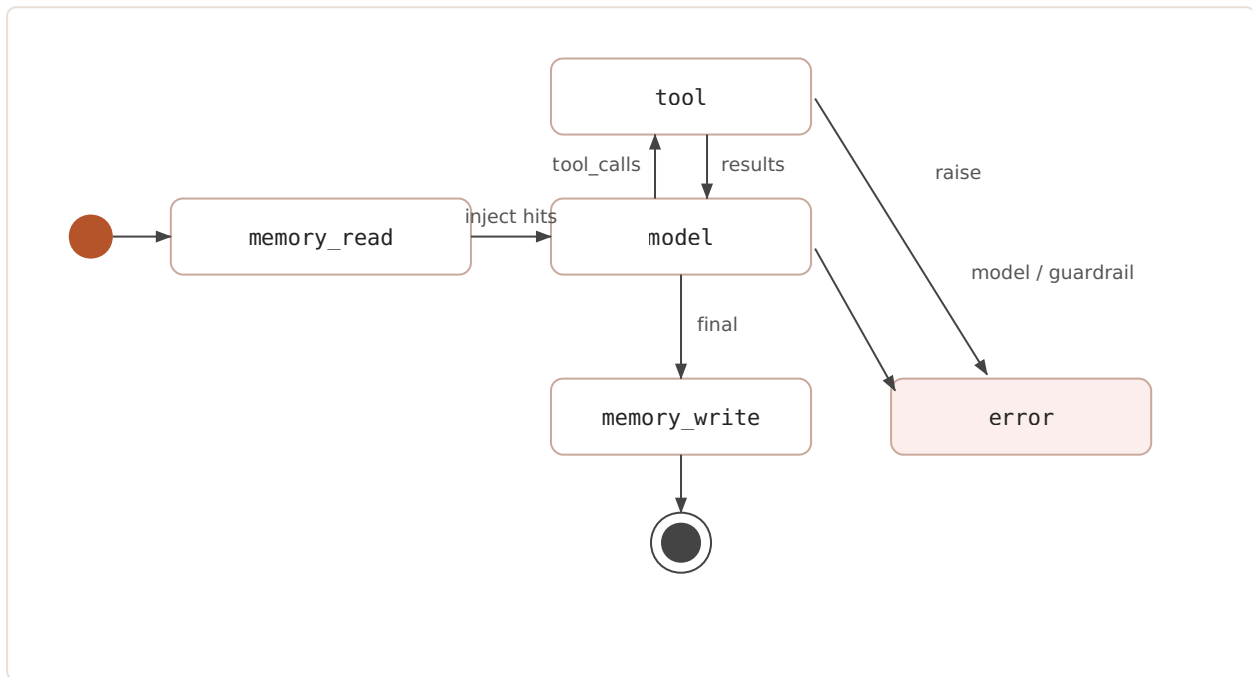
    # execution
    def run(self, input: str | Message, *, session_id: str | None = None,
            stream: bool = False, deadline_s: float | None = None) -> RunResult: ...
    async def arun(self, ...) -> RunResult | AsyncIterator[RunEvent]: ...

    # serving
    def serve(self, host: str = "127.0.0.1", port: int = 8080, *,
              mcp: bool = True, a2a: bool = True, metrics: bool = True) -> None: ...
    def asgi_app(self, **kw) -> "FastAPI": ... # for embedding/uvicorn programmatic
```

#### 3.1.2 Run loop (ReAct over a compiled graph)

The agent compiles to a 4-node graph; the executor drives it to a terminal state.





**Figure 2.** The agent run loop expressed as a state machine.

Node contracts:

| Node         | Input                          | Output                     | Side effects                  |
|--------------|--------------------------------|----------------------------|-------------------------------|
| memory_read  | state.messages                 | memory_hits                | vector search (scoped)        |
| model        | messages + hits + tool schemas | assistant msg / tool_calls | LLM call (generation span)    |
| tool         | pending_tool_calls             | tool result messages       | tool invoke (tool span)       |
| memory_write | final transcript               | —                          | async fact/summary extraction |
| error        | exception                      | RunResult(status=error)    | error span, checkpoint        |

Termination: final assistant message with no tool calls, or `step >= max_steps` (→ `MaxStepsExceeded`), or deadline exceeded (→ `DeadlineExceeded`).

### 3.1.3 State channels (agentkit/engine/state.py)

```

class AgentState(TypedDict):
    messages: Annotated[list[Message], add_messages]
    session_id: str
    step: int
    scratchpad: dict[str, Any]
    pending_tool_calls: list[ToolCall]
    memory_hits: list[MemoryRecord]
    status: Literal["running", "awaiting_tool", "done", "error", "interrupted"]
    error: ErrorInfo | None
  
```

`add_messages` is an append-merge reducer so concurrent writes (parallel flow branches) merge deterministically by message id.

### 3.1.4 Tool model & decorator

```

class Tool(BaseModel):
    name: str
    description: str
  
```

```

parameters: dict                # JSON Schema (draft 2020-12)
source: Literal["local", "mcp", "a2a"] = "local"
timeout_s: float = 30
handler: ToolHandler            # callable adapter (local fn / mcp call / a2a send)

async def invoke(self, args: dict, ctx: RunContext) -> ToolResult: ...

def tool(fn: Callable | None = None, *, name=None, timeout_s=30) -> Tool | Callable:
    """Infer JSON Schema from type hints + docstring; wrap as Tool."""

```

**ToolResult:**

```

class ToolResult(BaseModel):
    ok: bool
    content: Any                # JSON-serializable
    error: ErrorInfo | None = None
    latency_ms: float | None = None
    raw: Any | None = None      # provider raw, not sent to model

```

When `on_tool_error="surface"`, a failed tool yields `ToolResult(ok=False, error=...)` serialized back to the model so it can recover; `"raise"` aborts the run.

### 3.1.5 RunResult / streaming events

```

class RunResult(BaseModel):
    output: str
    messages: list[Message]
    tool_calls: list[ToolCallRecord]
    session_id: str
    status: Literal["done", "error", "interrupted"]
    usage: Usage                # tokens, cost
    trace_url: str | None
    error: ErrorInfo | None = None

# stream=True yields these:
RunEvent = TokenEvent | ToolStartEvent | ToolEndEvent | StepEvent | DoneEvent | ErrorEvent

```

## 3.2 Flow engine (agentkit/flow.py)

### 3.2.1 Builder API

```

class Flow:
    def __init__(self, name: str): ...
    def step(self, node: Agent | Callable | "Flow", *, name: str | None = None) -> "Flow": ...
    def when(self, predicate: Callable[[FlowState], bool]) -> "ConditionalBuilder": ...
    def parallel(self, *nodes) -> "Flow": ...          # fan-out / fan-in (merge reducer)
    def loop(self, body, *, until: Callable, max_iter: int = 10) -> "Flow": ...
    def map(self, body, *, over: str) -> "Flow": ...    # map over a list channel
    @classmethod
    def from_yaml(cls, path: str | Path) -> "Flow": ...
    def compile(self) -> CompiledGraph: ...
    def run(self, input, **kw) -> RunResult: ...
    def serve(self, **kw) -> None: ...

```

### 3.2.2 Declarative FlowSpec

```

class NodeSpec(BaseModel):
    id: str
    kind: Literal["agent", "function", "flow"]
    ref: str                # import path or registered name

```

```

    config: dict = {}

class EdgeSpec(BaseModel):
    src: str
    dst: str | list[str]          # list = parallel fan-out
    condition: str | None = None  # python expr over state

class FlowSpec(BaseModel):
    name: str
    nodes: list[NodeSpec]
    edges: list[EdgeSpec]
    entry: str
    finish: str | list[str]

```

Compilation maps `FlowSpec` → a `LangGraph StateGraph[FlowState]`; conditions compile to edge routers; `parallel` / `map` use the message/list merge reducers.

### 3.2.3 FlowState

```

class FlowState(TypedDict):
    input: Any
    node_outputs: Annotated[dict[str, Any], merge_dict]
    messages: Annotated[list[Message], add_messages]
    session_id: str
    cursor: str          # current node id (for resume)

```

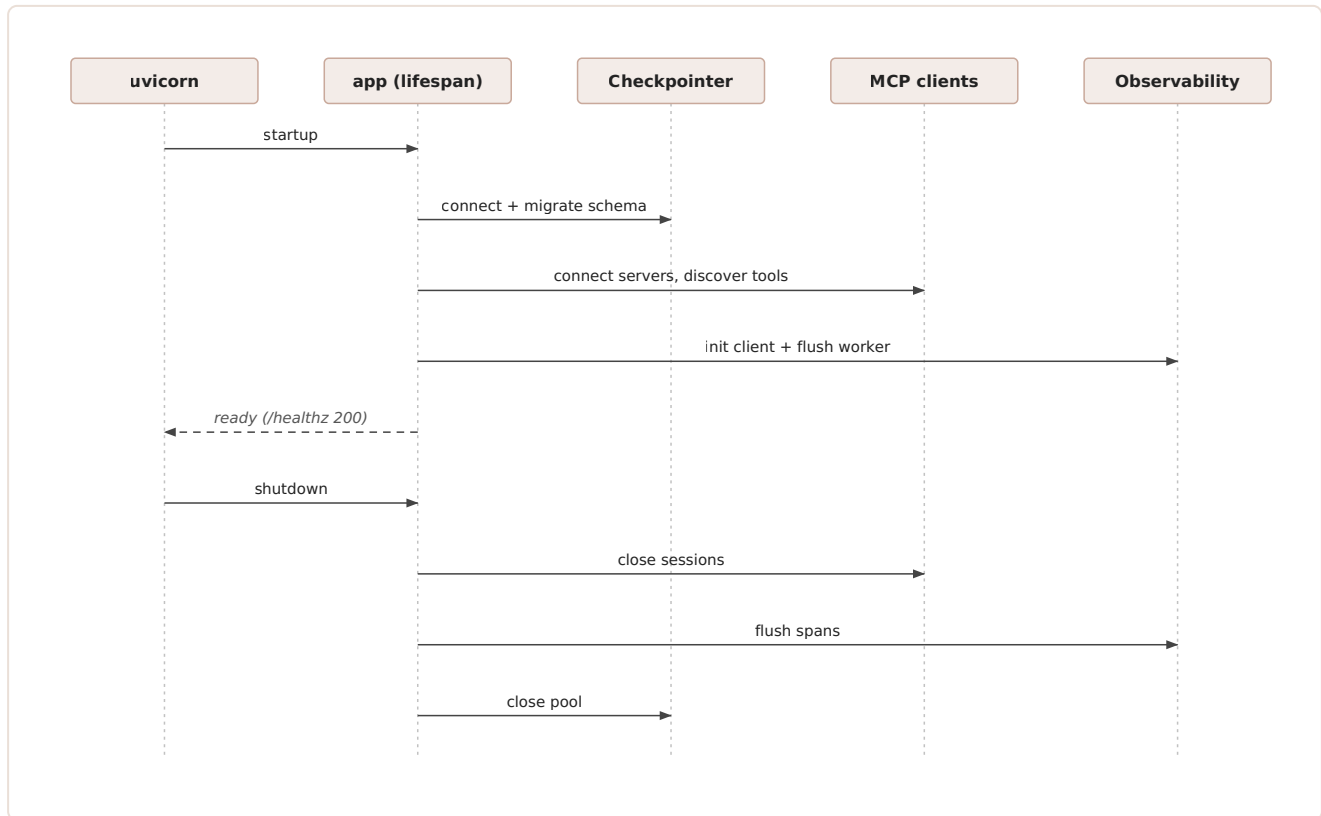
## 3.3 Runtime app (agentkit/runtime/app.py)

### 3.3.1 Factory & endpoints

```
def create_app(target: Agent | Flow, *, mcp=True, a2a=True, metrics=True) -> FastAPI: ...
```

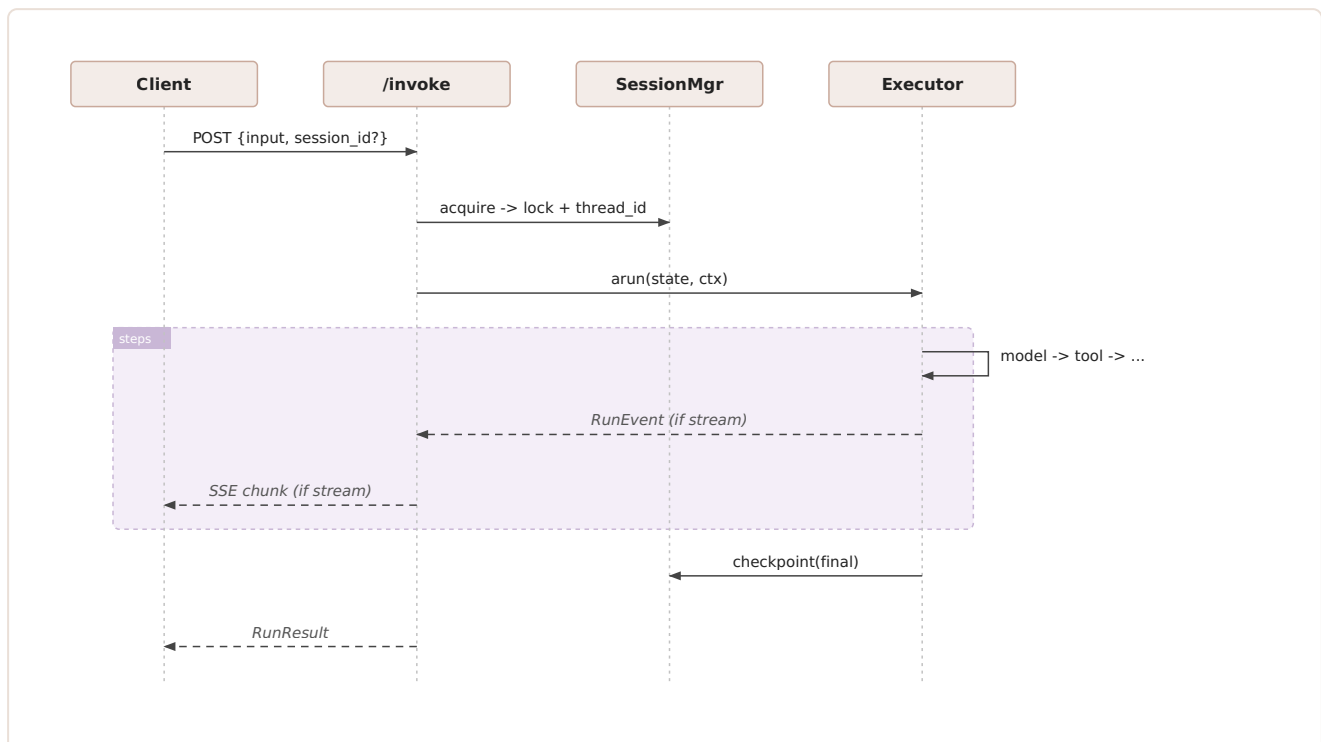
| Method · Path                  | Body / Params                              | Returns                        |
|--------------------------------|--------------------------------------------|--------------------------------|
| POST /invoke                   | {input, session_id?, stream?, deadline_s?} | RunResult or text/event-stream |
| GET /runs/{session_id}         | —                                          | last RunResult + status        |
| POST /runs/{session_id}/resume | {}                                         | resumes from checkpoint        |
| POST /mcp                      | MCP Streamable HTTP (JSON-RPC)             | tool list / tool results       |
| GET /.well-known/agent.json    | —                                          | <b>A2A Agent Card</b>          |
| POST /a2a                      | A2A tasks/send, tasks/get                  | task object / SSE stream       |
| GET /health, GET /healthz      | —                                          | liveness/readiness             |
| GET /metrics                   | —                                          | Prometheus exposition          |

### 3.3.2 Lifespan



**Figure 3.** Runtime application lifespan: startup and shutdown.

### 3.3.3 Invoke handler (sync + streaming)



**Figure 4.** Invoke handler with optional token streaming.

### 3.4 Checkpointer (agentkit/runtime/checkpoint.py)

```
class Checkpointer(Protocol):
    async def put(self, thread_id: str, cp: Checkpoint) -> str: ...
    async def get(self, thread_id: str, checkpoint_id: str | None = None) -> Checkpoint | None: ...
    async def list(self, thread_id: str, limit: int = 50) -> list[CheckpointMeta]: ...
    async def delete(self, thread_id: str) -> None: ...

class SqliteCheckpointer(Checkpointer): ... # default: file or :memory:
class PostgresCheckpointer(Checkpointer): ... # prod
```

Schema (SQLite/Postgres compatible):

```
CREATE TABLE checkpoints (
    thread_id      TEXT NOT NULL,
    checkpoint_id  TEXT NOT NULL,
    parent_id      TEXT,
    state          BLOB NOT NULL,          -- msgpack-serialized AgentState/FlowState
    metadata       JSON NOT NULL,         -- {step, status, node, ts}
    created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (thread_id, checkpoint_id)
);
CREATE INDEX ix_cp_thread ON checkpoints(thread_id, created_at DESC);

CREATE TABLE checkpoint_writes (        -- pending channel writes for exactly-once resume
    thread_id      TEXT NOT NULL,
    checkpoint_id  TEXT NOT NULL,
    task_id        TEXT NOT NULL,
    idx            INTEGER NOT NULL,
    channel        TEXT NOT NULL,
    value          BLOB NOT NULL,
    PRIMARY KEY (thread_id, checkpoint_id, task_id, idx)
);
```

Resume contract: on `resume`, load latest checkpoint, replay any `checkpoint_writes` not yet folded into state, then continue the graph from `metadata.node`. Serialization is msgpack with a typed codec registry; unknown types fail closed (`CheckpointError`).

### 3.5 Session manager (agentkit/runtime/session.py)

```
class SessionManager:
    def __init__(self, max_concurrent: int = 64): ...
    async def acquire(self, session_id: str | None) -> Session: ... # creates thread_id
    async def release(self, session: Session) -> None: ...
    def lock(self, session_id: str) -> asyncio.Lock: ...           # 1 active run / session
```

- **Isolation** without VMs: each session is an asyncio task with its own `thread_id`, `RunContext`, and checkpoint namespace. No shared mutable state beyond read-only registries.
- **Concurrency**: a global semaphore bounds concurrent runs; a per-session lock serializes writes to one thread. CPU-bound tool work → `run_in_executor` (thread/process pool); risky code → subprocess or E2B (never on the event loop).

## 4 Protocol modules

### OVERVIEW

Specifies how an agent both consumes and exposes the two interoperability protocols. MCP turns external services into tools and the agent's own tools into a server; A2A lets the agent act as, and call, peer agents. Together they make the platform a participant in a wider agent ecosystem.

### 4.1 MCP client (agentkit/protocols/mcp\_client.py)

```
class MCPClient:
    def __init__(self, url: str, *, transport: Literal["stdio","http","sse"]="http",
                 auth: AuthConfig | None = None, namespace: str | None = None): ...
    async def connect(self) -> None: ...
    async def list_tools(self) -> list[MCPToolDescriptor]: ...
    async def call_tool(self, name: str, args: dict, *, ctx: RunContext) -> ToolResult: ...
    async def close(self) -> None: ...
    def as_tools(self) -> list[Tool]: ...      # adapt MCP tools -> agent Tool objects
```

- Discovery on connect; tools namespaced ( `{namespace}.{tool}` ) to avoid collisions.
- Auth: bearer/api-key/OAuth2 client-credentials resolved via `CredentialResolver`.
- Reconnect with backoff; tool schema cache invalidated on `notifications/tools/list_changed`.
- Each `call_tool` opens a `tool_call` span with server, name, latency, ok/error.

### 4.2 MCP server (agentkit/protocols/mcp\_server.py)

```
class MCPServer:
    def __init__(self, name: str, version: str = "0.1.0"): ...
    def tool(self, fn=None, *, name=None): ...      # decorator -> registers + schema
    def resource(self, uri_template: str): ...      # optional MCP resources
    def serve(self, host="127.0.0.1", port=9000, transport="http") -> None: ...
    def asgi_app(self) -> "Starlette": ...          # mountable under agent's FastAPI /mcp
```

Built on FastMCP. When an `Agent.serve(mcp=True)`, the runtime mounts an auto-generated MCP server exposing (a) the agent's local tools and (b) an `invoke_agent` meta-tool that runs the agent itself — letting any MCP client use the agent as a single tool.

### 4.3 A2A (agentkit/protocols/a2a.py)

```
class A2AServer:
    def __init__(self, agent: Agent, *, card: AgentCard | None = None): ...
    def asgi_app(self) -> "Starlette": ...          # /.well-known/agent.json + /a2a
    # task lifecycle: submitted -> working -> (completed | failed | canceled)

class A2AClient:
    async def send(self, peer_url: str, message: Message, *, stream=False)
        -> A2ATask | AsyncIterator[A2AEvent]: ...
    async def get(self, peer_url: str, task_id: str) -> A2ATask: ...
```

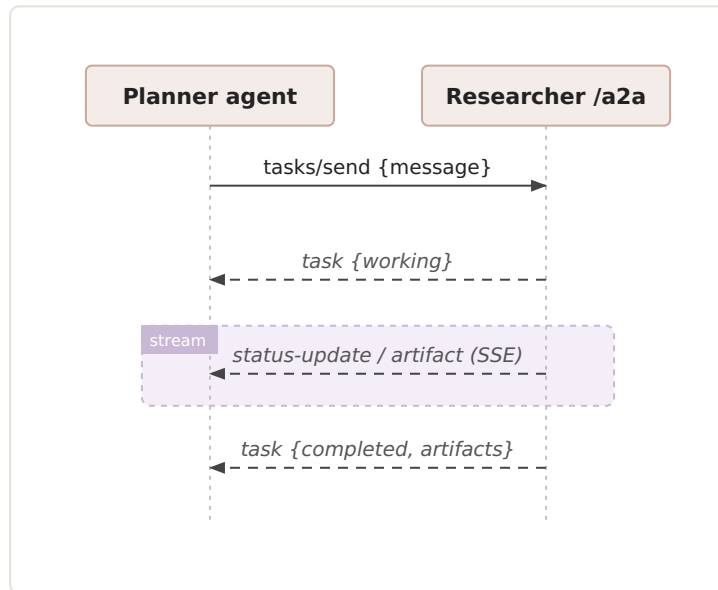
`AgentCard` (published at `/.well-known/agent.json`):

```

class AgentCard(BaseModel):
    name: str
    description: str
    url: str
    version: str
    capabilities: Capabilities          # streaming, push, state-transition history
    skills: list[Skill]                 # id, name, description, examples
    auth: AuthSchemes

```

A2A peers listed on an Agent become callable tools ( `source="a2a"` ): the model "calls" the peer as a tool; under the hood it's `tasks/send` with progress streamed into the tool span.



**Figure 5.** A2A task interaction between a planner and a served agent.

# 5 Memory (agentkit/memory/)

## OVERVIEW

Details the memory subsystem: one store interface, three partition scopes, and the extraction strategies that populate long-term memory. It fixes the read and write paths so recall happens before the model call and persistence happens off the critical path.

## 5.1 Interface & scopes

```
class MemoryStore(Protocol):
    async def add(self, items: list[Message] | Fact, *, scope: Scope, ctx: RunContext) -> None: ...
    async def search(self, query: str, *, scope: Scope, k: int = 6) -> list[MemoryRecord]: ...
    async def get_session(self, session_id: str) -> list[Message]: ...
    async def summarize(self, session_id: str) -> Summary: ...
    async def forget(self, *, scope: Scope, filter: dict) -> int: ...
```

Scope = Literal["session", "user", "agent"] # short-term / per-user / global long-term

## 5.2 Strategies (Mem0-backed)

| Strategy        | Trigger                          | Stored artifact          |
|-----------------|----------------------------------|--------------------------|
| semantic        | post-run extraction              | atomic facts (embedded)  |
| summary         | session length / token threshold | rolling session summary  |
| user_preference | detected preference statements   | typed preference records |

```
class MemoryConfig(BaseModel):
    backend: Literal["chroma", "sqlite_vec", "qdrant", "pgvector"] = "sqlite_vec"
    strategies: list[Literal["semantic", "summary", "user_preference"]] = ["semantic", "summary"]
    embedder: str = "default"
    top_k: int = 6
    write_async: bool = True # extraction off the hot path
```

## 5.3 Records & read/write paths

```
class MemoryRecord(BaseModel):
    id: str
    scope: Scope
    text: str
    metadata: dict # {session_id, user_id, kind, ts}
    score: float | None = None # set on search
```

- **Read path** (`memory_read` node): scoped vector search; top-k injected as a system context block before the model call.
- **Write path** (`memory_write` node): if `write_async`, enqueue extraction to a background task so the run returns immediately; failures are logged, never block the run.
- Embedded backends (`sqlite_vec`, `chroma`) need no server; `qdrant` / `pgvector` for scale.



## 6 Tools (agentkit/tools/)

### OVERVIEW

Covers the three execution capabilities — browser, code, and desktop — and the trust boundary each sits behind. It defines the interfaces an author codes against and the policy that decides, per call, whether work runs in-process, in a subprocess, or in a managed sandbox.

### 6.1 Browser (tools/browser.py)

```
class BrowserTool:
    def __init__(self, *, headless=True, profile: str | None=None,
                  backend: Literal["local","steel"]="local"): ...
    # exposed sub-tools (as an MCP-style group or native tools):
    #   navigate(url) · click(index) · type(text) · extract(schema) ·
    #   screenshot() · scroll(dir) · back() · tabs()
    async def run_task(self, instruction: str, *, ctx: RunContext) -> BrowserResult: ...
```

Wraps browser-use over a local Chromium (Playwright). No VM. `backend="steel"` swaps in a self-hosted/remote browser API for parallel sessions. DOM is distilled to indexed interactive elements before being handed to the model.

### 6.2 Code execution (tools/code.py)

```
class CodeExecutor(Protocol):
    async def run(self, code: str, *, lang="python", timeout_s=30,
                  files: dict[str,bytes] | None=None, network=False) -> ExecResult: ...

class SubprocessExecutor(CodeExecutor):    # trusted: rlimit cpu/mem, temp cwd, optional net off
    ...
class E2BExecutor(CodeExecutor):           # untrusted: managed cloud sandbox (no VM you run)
    ...

class ExecResult(BaseModel):
    stdout: str; stderr: str; exit_code: int
    artifacts: dict[str, bytes] = {}
    timed_out: bool = False
```

Selection policy: `trust="trusted"` → `SubprocessExecutor` (host process, resource-limited); `trust="untrusted"` → `E2BExecutor`. Default is configurable; untrusted defaults to E2B if configured, else raises `SandboxUnavailable` (fail closed).

### 6.3 Computer / desktop use (tools/computer.py)

```
class ComputerDriver(Protocol):
    async def screenshot(self) -> bytes: ...
    async def click(self, x: int, y: int, button="left") -> None: ...
    async def type(self, text: str) -> None: ...
    async def key(self, combo: str) -> None: ...

class E2BDesktopDriver(ComputerDriver): ...    # remote managed desktop (preferred, no local VM)
class PyAutoGuiDriver(ComputerDriver): ...    # host control, UNSANDBOXED, opt-in only
```

Disabled unless `enable_computer_use=True`. `PyAutoGuiDriver` drives the host machine and is single-user only; multi-tenant or untrusted use must select `E2BDesktopDriver`.

## 6.4 Tool registry & plugins

```
class ToolRegistry:
    def register(self, tool: Tool) -> None: ...
    def get(self, name: str) -> Tool: ...
    def discover_entrypoints(self) -> None: ... # group: "agentkit.tools"
```

Third parties ship tools as packages exposing the `agentkit.tools` entry point; they appear by name in any agent spec.

# 7 Observability & evaluation (agentkit/observability/)

## OVERVIEW

Describes the single hub that makes runs inspectable and measurable. It fixes a trace and span taxonomy so every run is structured identically, and an evaluation runner that scores quality offline and online and writes results back to the same place.

## 7.1 Trace/span taxonomy

One **trace per run**; nested spans mirror the engine:

```

trace: run(session_id)
├ span: agent_step[i]
│   ├── generation: model_call      (model, prompt, completion, tokens, cost, latency)
│   ├── span: memory_read          (query, hits, k)
│   └── span: tool_call[name]       (args, result.ok, latency, source=local|mcp|a2a)
├ span: a2a_task[peer]              (task_id, status transitions)
└ span: memory_write                (facts_extracted, summary_updated)

```

```

class Tracer:
    def trace(self, name: str, **attrs) -> "TraceCtx": ...
    def span(self, name: str, **attrs) -> "SpanCtx": ...
    def generation(self, name: str, **attrs) -> "GenCtx": ... # LLM-specific
    def score(self, trace_id: str, name: str, value: float, comment: str | None=None) -> None: ...
    def flush(self) -> None: ...

```

- Backed by Langfuse; OpenTelemetry exporter optional for Grafana/Tempo fan-out.
- Instrumentation is implicit: the engine wraps nodes via `@observed`; authors write no tracing code. Sampling configurable (`OBS_SAMPLE_RATE`), default 1.0 in dev.
- `RunResult.trace_url` deep-links the Langfuse trace.

## 7.2 Evaluation runner (observability/eval.py)

```

class EvalRunner:
    def __init__(self, target: Agent | Flow, *, dataset: str | Dataset,
                 metrics: list[Metric]): ...
    async def run(self, *, sample: int | None=None) -> EvalReport: ... # pushes scores to Langfuse

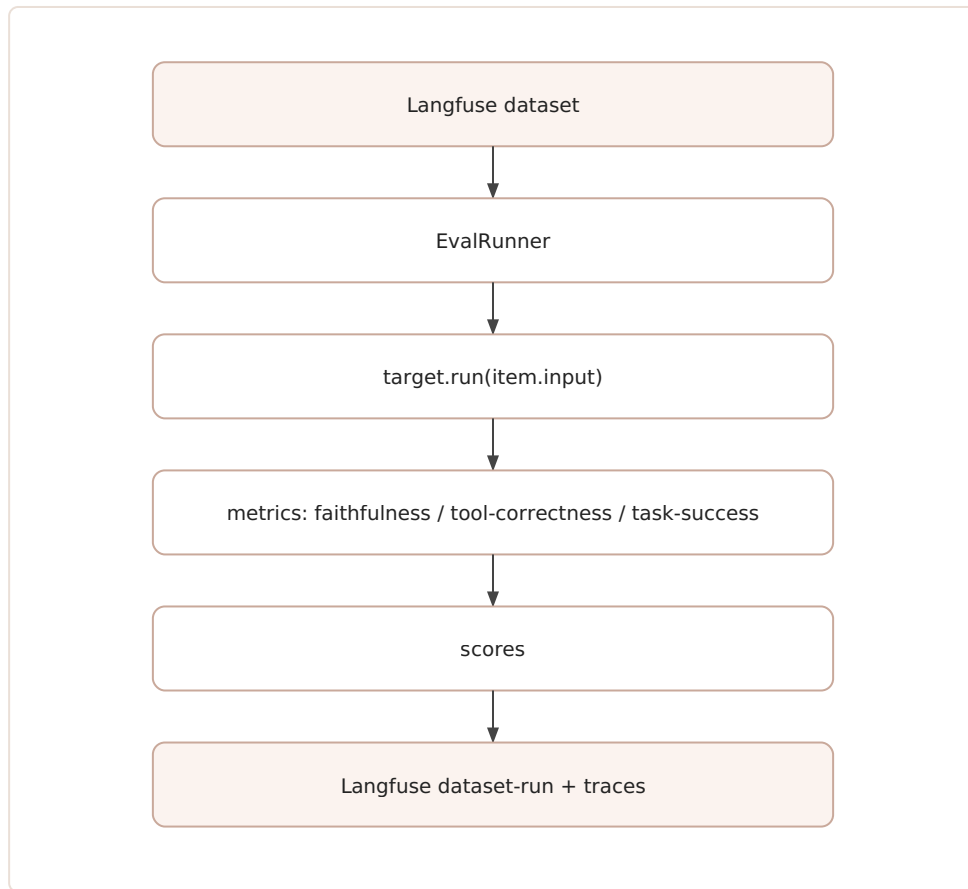
# built-in metrics (wrap DeepEval / Ragas / custom):
class Metric(Protocol):
    name: str
    async def score(self, item: EvalItem, output: RunResult) -> Score: ...

# provided:
Faithfulness() · AnswerRelevancy() · ContextPrecision() ·
ToolCorrectness() · TaskSuccess(judge="llm") · Latency() · Cost()

```

- **Datasets** live in Langfuse (`dataset` name resolves to a Langfuse dataset); each item is an input + optional expected output/context.
- **Offline**: CI runs `EvalRunner.run`; scores attach to a Langfuse dataset-run for diffing across versions.

- **Online:** a sampling hook scores live traces with LLM-as-judge and writes `score`s back to the same trace, enabling production quality dashboards.



**Figure 6.** Offline and online evaluation pipeline.

## 8 Identity & secrets (agentkit/identity/)

### OVERVIEW

Defines how the platform obtains credentials without requiring identity infrastructure of its own. It separates inbound exposure auth from outbound tool auth and keeps secrets out of traces.

```
class SecretProvider(Protocol):
    def get(self, key: str) -> str | None: ...

class EnvProvider(SecretProvider): ...
class KeyringProvider(SecretProvider): ...
class VaultProvider(SecretProvider): ...      # optional, no infra required by core

class CredentialResolver:
    """Resolve outbound auth per tool/server (api key, oauth token, basic)."""
    def resolve(self, target: str) -> AuthConfig | None: ...
```

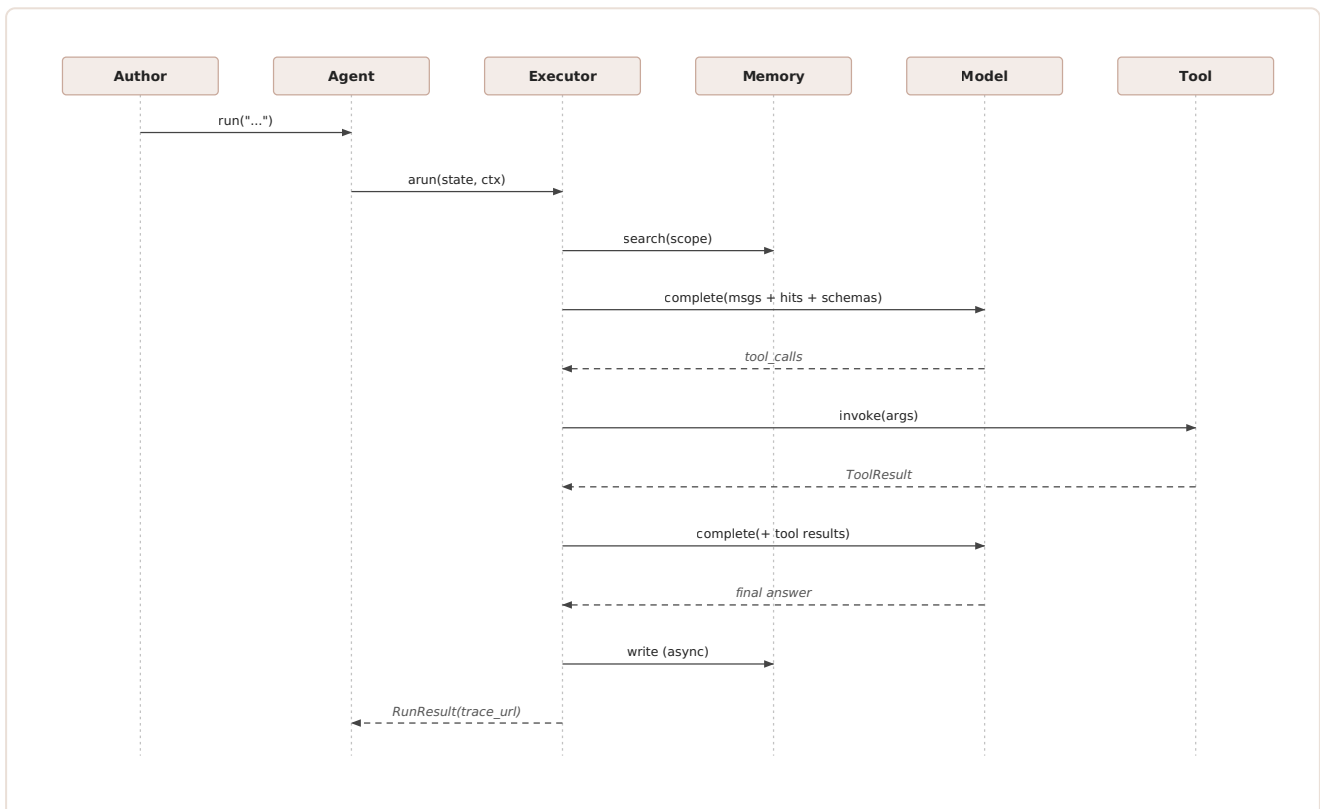
- Core needs no identity infra: model + tool keys come from env/keyring.
- Outbound tool auth is keyed by MCP server / A2A peer / tool name.
- Inbound auth on `serve` is opt-in middleware (bearer/OAuth resource-server) for when the app is exposed beyond localhost.

## 9 Execution lifecycle (end-to-end sequences)

### OVERVIEW

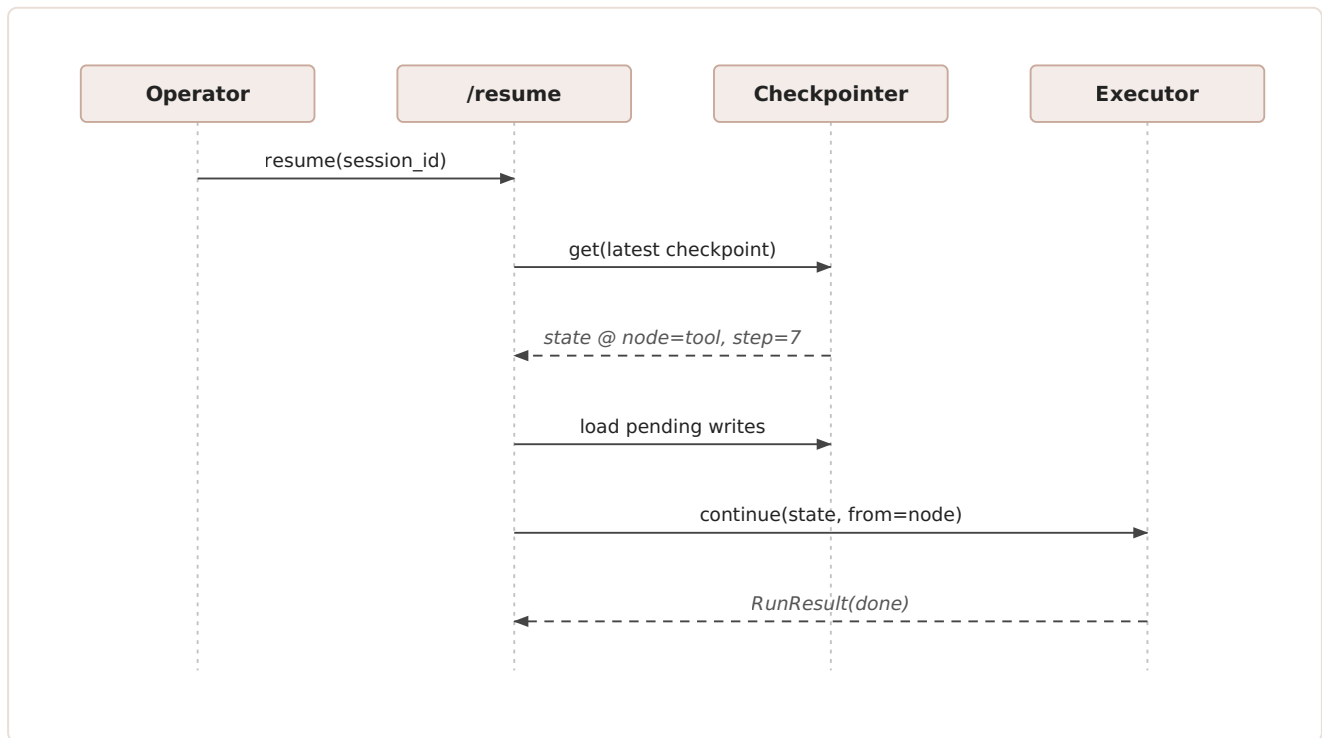
Walks the system end to end through its two most important flows — a normal in-process run and a durable resume after a crash — tying together the engine, memory, model, tools, tracer, and checkpointer specified earlier.

### 9.1 In-process run (Agent.run)



**Figure 7.** End-to-end in-process run.

## 9.2 Durable resume after crash



**Figure 8.** Durable resume after a process crash.

# 10 Error handling

## OVERVIEW

Gives every failure a typed name, a retry disposition, and a defined effect on the run. The taxonomy and policy table make error behavior deterministic and auditable rather than incidental.

### 10.1 Taxonomy

```
class AgentError(Exception): ...           # base; carries ErrorInfo
class ModelError(AgentError): ...
class RateLimitError(ModelError): ...     # transient -> retry
class ToolError(AgentError): ...
class ToolTimeout(ToolError): ...
class MCPConnectionError(AgentError): ... # transient -> retry/reconnect
class A2ATaskFailed(AgentError): ...
class MemoryError_(AgentError): ...       # non-fatal -> degrade
class CheckpointError(AgentError): ...    # fatal
class SandboxUnavailable(ToolError): ...  # fail closed
class GuardrailViolation(AgentError): ... # deterministic -> no retry
class MaxStepsExceeded(AgentError): ...
class DeadlineExceeded(AgentError): ...

class ErrorInfo(BaseModel):
    type: str; message: str; retrieable: bool
    where: str                               # node/tool/server
    cause: str | None = None
```

### 10.2 Policy

| Class                               | Retry            | Backoff    | Run outcome                      |
|-------------------------------------|------------------|------------|----------------------------------|
| RateLimitError, MCPConnectionError  | yes ( $\leq 3$ ) | exp+jitter | continue on success              |
| ToolTimeout                         | configurable     | once       | surface to model or raise        |
| ToolError (logic)                   | no               | —          | surface to model (on_tool_error) |
| MemoryError_                        | no               | —          | degrade: skip memory, continue   |
| GuardrailViolation, CheckpointError | no               | —          | terminate run (status=error)     |
| MaxStepsExceeded / DeadlineExceeded | no               | —          | terminate, checkpoint preserved  |

Transient retries use `tenacity`-style decorators; every retry is a span event. Failures always write a final checkpoint so a run is resumable or auditable.



# 11 Security model

---

## OVERVIEW

States the trust boundaries explicitly, including the consequence of running without VMs: untrusted execution is delegated rather than pretended away. It also covers secret redaction, default-local binding, and the prompt-injection surface.

- **Trust boundaries:** browser-use (sandboxed subprocess) < trusted code (resource-limited subprocess) < untrusted code (E2B managed sandbox) < desktop use (E2B desktop, or host via pyautogui — explicitly unsafe).
- **No VM** ⇒ no hardware isolation in-process; untrusted execution is delegated to E2B. This is a deliberate, documented boundary (NFR/Non-goal N1).
- **Secrets** never enter traces; the tracer redacts configured keys and tool `raw` payloads.
- **Inbound exposure:** `serve` binds `127.0.0.1` by default; non-local binding requires auth middleware to be enabled.
- **Prompt-injection surface:** tool outputs and web content are untrusted input; guardrail hooks (`pre_model`, `post_tool`) can filter before content reaches the model.

# 12 Configuration reference

## OVERVIEW

The operator-facing surface: every environment variable, and the complete declarative specifications for an agent and a flow. These are the contracts a deployment is configured against.

### 12.1 Environment variables

| Var                                                       | Default                 | Purpose                                    |
|-----------------------------------------------------------|-------------------------|--------------------------------------------|
| AGENTKIT_DB_URL                                           | sqlite:///./agentkit.db | checkpointer DSN                           |
| AGENTKIT_LOG_LEVEL                                        | INFO                    | logging                                    |
| AGENTKIT_MAX_CONCURRENT                                   | 64                      | concurrent runs ceiling                    |
| AGENTKIT_OBS                                              | langfuse                | observability backend (langfuse/otel/none) |
| LANGFUSE_PUBLIC_KEY / LANGFUSE_SECRET_KEY / LANGFUSE_HOST | —                       | Langfuse                                   |
| OBS_SAMPLE_RATE                                           | 1.0                     | trace sampling                             |
| MEMORY_BACKEND                                            | sqlite_vec              | memory store                               |
| E2B_API_KEY                                               | —                       | untrusted sandbox / desktop                |
| <PROVIDER>_API_KEY                                        | —                       | model keys (e.g. ANTHROPIC_API_KEY)        |

### 12.2 Full agent spec (YAML)

```
apiVersion: agentkit/v1
kind: Agent
metadata:
  name: researcher
spec:
  model: claude-opus-4-8
  instructions: |
    You research a topic and produce a sourced brief.
  max_steps: 25
  step_timeout_s: 120
  on_tool_error: surface

  tools:
    - ref: search_db          # local, from agentkit.tools entrypoint
    - ref: browser            # built-in browser group
      config: { headless: true, backend: local }

  mcp_servers:
    - url: http://localhost:9000/mcp
      namespace: billing
      auth: { type: bearer, secret: BILLING_TOKEN }

  a2a_peers:
    - http://writer:8081/.well-known/agent.json
```

```
memory:
  backend: sqlite_vec
  strategies: [semantic, summary, user_preference]
  top_k: 6
  write_async: true

observability: langfuse

runtime:
  serve: { host: 127.0.0.1, port: 8080, mcp: true, a2a: true, metrics: true }
```

## 12.3 Flow spec (YAML)

```
apiVersion: agentkit/v1
kind: Flow
metadata: { name: research-and-write }
spec:
  entry: researcher
  finish: reviewer
  nodes:
    - { id: researcher, kind: agent, ref: ./researcher.yaml }
    - { id: writer,      kind: agent, ref: ./writer.yaml }
    - { id: reviewer,   kind: agent, ref: ./reviewer.yaml }
  edges:
    - { src: researcher, dst: writer }
    - { src: writer, dst: reviewer, condition: "state.node_outputs['writer'].needs_review" }
    - { src: writer, dst: __end__, condition: "not state.node_outputs['writer'].needs_review" }
```

# 13 Packaging & distribution

## OVERVIEW

Defines how the library is built, what optional dependency groups exist, the command-line surface, and how wire and storage contracts are versioned so upgrades stay safe.

### 13.1 pyproject.toml (excerpt)

```
[project]
name = "agentkit"
requires-python = ">=3.11"
dependencies = ["pydantic>=2", "langgraph", "fastapi", "uvicorn",
               "httpx", "mcp", "a2a-sdk", "mem0ai", "langfuse", "msgpack"]

[project.optional-dependencies]
browser = ["browser-use", "playwright"]
sandbox = ["e2b"]
eval = ["deepeval", "ragas"]
qdrant = ["qdrant-client"]
postgres = ["psycopg[binary]"]
all = ["agentkit[browser,sandbox,eval,qdrant,postgres]"]

[project.scripts]
agentkit = "agentkit.cli:main"

[project.entry-points."agentkit.tools"]
# third-party tools register here

[project.entry-points."agentkit.backends"]
# third-party stores/checkpointers/executors register here
```

### 13.2 CLI

```
agentkit run <module:agent> --input "." [--session ID] [--stream]
agentkit serve <module:agent|flow> [--host] [--port] [--no-mcp] [--no-a2a]
agentkit eval <module:target> --dataset NAME --metrics faithfulness,tool_correctness
agentkit resume <session_id>
```

### 13.3 Versioning

SemVer. Wire contracts (MCP/A2A payloads, checkpoint codec, agent spec `apiVersion`) are versioned independently; checkpoint codec carries a schema version and refuses to load newer versions (forward-incompatible → `CheckpointError`).

# 14 Extensibility points

## OVERVIEW

Enumerates the seams where third parties extend the platform — tools, stores, checkpointers, executors, guardrails, engine and observability backends — without forking core.

| Extension         | Mechanism                                                 |
|-------------------|-----------------------------------------------------------|
| New tool          | @tool function or agentkit.tools entry point              |
| New memory store  | implement MemoryStore, register agentkit.backends         |
| New checkpointer  | implement Checkpointer, register agentkit.backends        |
| New code executor | implement CodeExecutor                                    |
| Guardrails        | pre_model / post_tool hooks on Agent                      |
| Engine backend    | swap LangGraph for Strands behind CompiledGraph interface |
| Obs backend       | implement Tracer (Langfuse default, OTel alt)             |

# 15 Build milestones

## OVERVIEW

Sequences the implementation into seven phases, each with a single clear exit criterion, so progress is measurable and every phase yields something runnable.

| Phase        | Deliverable                                  | Exit criteria                                      |
|--------------|----------------------------------------------|----------------------------------------------------|
| M1 Spine     | Agent + @tool + engine + SQLite checkpointer | run() completes a tool-using task in-process       |
| M2 Serve     | FastAPI factory + /invoke (+stream)          | served agent answers over HTTP                     |
| M3 Protocols | MCP client/server + A2A                      | agent consumes & exposes MCP; calls an A2A peer    |
| M4 Memory    | Mem0 adapter + embedded store                | session + long-term recall verified                |
| M5 Obs+Eval  | Langfuse tracing + EvalRunner                | traces visible; dataset run scores in Langfuse     |
| M6 Tools     | browser-use; code exec; computer (flagged)   | browser task + sandboxed code run                  |
| M7 Flow+Pkg  | Flow graph, YAML, CLI, PyPI                  | pip install; another project builds an agent on it |

# 16 Glossary

---

## OVERVIEW

Defines the recurring terms of art precisely so the document reads consistently.

- **Checkpoint** — serialized graph state + pending writes enabling exactly-once resume.
- **Generation span** — an LLM-call span carrying prompt/completion/tokens/cost.
- **Scope** — memory partition: `session` (short-term), `user`, `agent` (long-term).
- **Surface vs raise** — whether a tool failure returns to the model or aborts the run.
- **Trust boundary** — the point at which execution is delegated to an isolated sandbox (E2B).

# 17 Open questions

---

## OVERVIEW

Records the design decisions deliberately left open, so they are resolved on purpose rather than by accident during implementation.

1. Default untrusted-code behavior when E2B is absent — hard fail (current) vs. degrade to trusted subprocess with a loud warning?
2. A2A peer-as-tool: expose each remote **skill** as a distinct tool, or one `delegate` tool per peer? (Granularity vs. prompt bloat.)
3. Memory write-conflict policy across parallel flow branches writing the same `user` scope.
4. Streaming semantics for nested flows (per-node SSE multiplexing vs. flattened stream).